RAJALAKSHMI ENGINEERING COLLEGE

(Autonomous)

RAJALAKSHMI NAGAR, THANDALAM, CHENNAI-602105

EXPT NO: 5	DATE: _____

A PYTHON PROGRAM TO SOLVE DYNAMIC PROGRAMMING PROBLEMS

AIM:

To implement Dynamic Programming techniques – 0/1 Knapsack, Subset Sum Problem, and Longest Common Subsequence – to solve classical optimization problems.

PROCEDURE:

1. Understand the optimal substructure and overlapping subproblems property.
2. Implement a bottom-up DP table for 0/1 Knapsack Problem.
3. Implement Subset Sum Problem to check if a target sum exists.
4. Implement Longest Common Subsequence to find length of common subsequence.
5. Display the optimal solution for each problem.

PROBLEM 1: 0/1 Knapsack – Weights=[3,4,5,6], Values=[4,5,3,7], Capacity=10

CODE

```
def knapsack_01(weights, values, W):
n = len(weights)
dp = [[0]*(W+1) for _ in range(n+1)]
for i in range(1, n+1):
for w in range(W+1):
dp[i][w] = dp[i-1][w]
if weights[i-1] <= w:
dp[i][w] = max(dp[i][w], values[i-1] + dp[i-1][w - weights[i-1]])
chosen = []; w = W
for i in range(n, 0, -1):
if dp[i][w] != dp[i-1][w]:
chosen.append(i); w -= weights[i-1]
return dp[n][W], chosen[::-1]
weights=[3,4,5,6]; values=[4,5,3,7]; W=10
max_val, items = knapsack_01(weights, values, W)
print("Maximum Value:", max_val)
print("Items selected (1-indexed):", items)
```

OUTPUT

```
Maximum Value: 12
Items selected (1-indexed): [1, 4]
```

PROBLEM 2: Subset Sum Problem

CODE

```
def subset_sum(arr, sum):
```

```
n = len(arr)
dp = [[False for _ in range(sum + 1)] for _ in range(n + 1)]
for i in range(n + 1):
dp[i][0] = True
for i in range(1, n + 1):
for s in range(1, sum + 1):
if arr[i-1] <= s:
dp[i][s] = dp[i-1][s] or dp[i-1][s-arr[i-1]]
else:
dp[i][s] = dp[i-1][s]
return dp[n][sum]
arr = [3, 34, 4, 12, 5, 2]
sum = 9
print(subset_sum(arr, sum))
```

OUTPUT

```
True
```

PROBLEM 3: Longest Common Subsequence Problem

CODE

```
def lcs(X, Y):
m = len(X)
n = len(Y)
dp = [[0 for _ in range(n + 1)] for _ in range(m + 1)]
for i in range(m + 1):
for j in range(n + 1):
if i == 0 or j == 0:
dp[i][j] = 0
elif X[i-1] == Y[j-1]:
dp[i][j] = dp[i-1][j-1] + 1
else:
dp[i][j] = max(dp[i-1][j], dp[i][j-1])
return dp[m][n]
X = "AGGTAB"
Y = "GXTXAYB"
print(lcs(X, Y))
```

OUTPUT

```
4
```

RESULT:

Thus, Dynamic Programming problems – 0/1 Knapsack, Subset Sum, and Longest Common Subsequence – have been implemented and executed successfully using Python.